

Los tres ejercicios del día del primer bloque, resueltos

Qué es esta hoja. Los tres ejercicios del día de las hojas de los días 02, 03 y 04, juntos y con la solución desarrollada: el pase forward completo por un MLP, la capa de *BatchNorm* sobre un minibatch y la convolución con *pooling* a mano. Son los tres obligatorios del primer bloque, y son los que entran en el examen con el cálculo incluido. El resto de ejercicios de esas tres hojas es ampliación: de ellos sólo puede preguntarse la idea, a nivel conceptual.

Cómo usarla. Tapa la caja de solución, haz el ejercicio entero en papel y compara después. Los enunciados y las soluciones son idénticos a los de las hojas originales; esta hoja sólo los reúne.

EL EJERCICIO DEL DÍA obligatorio · entra en el examen

1. Pase forward completo por una red $2 \rightarrow 3 \rightarrow 1$ y error final

Día 02 · Tema 1 — neurona, MLP y backpropagation.

Un operador de telecomunicaciones estima la probabilidad de fuga (*churn*) de cada cliente con una red de dos entradas, una capa oculta de tres neuronas con activación ReLU y una neurona de salida con activación sigmoide. Las dos entradas están estandarizadas sobre la cartera de clientes (media 0, desviación típica 1): x_1 es el gasto mensual y x_2 el número de llamadas al servicio de soporte en el último trimestre.

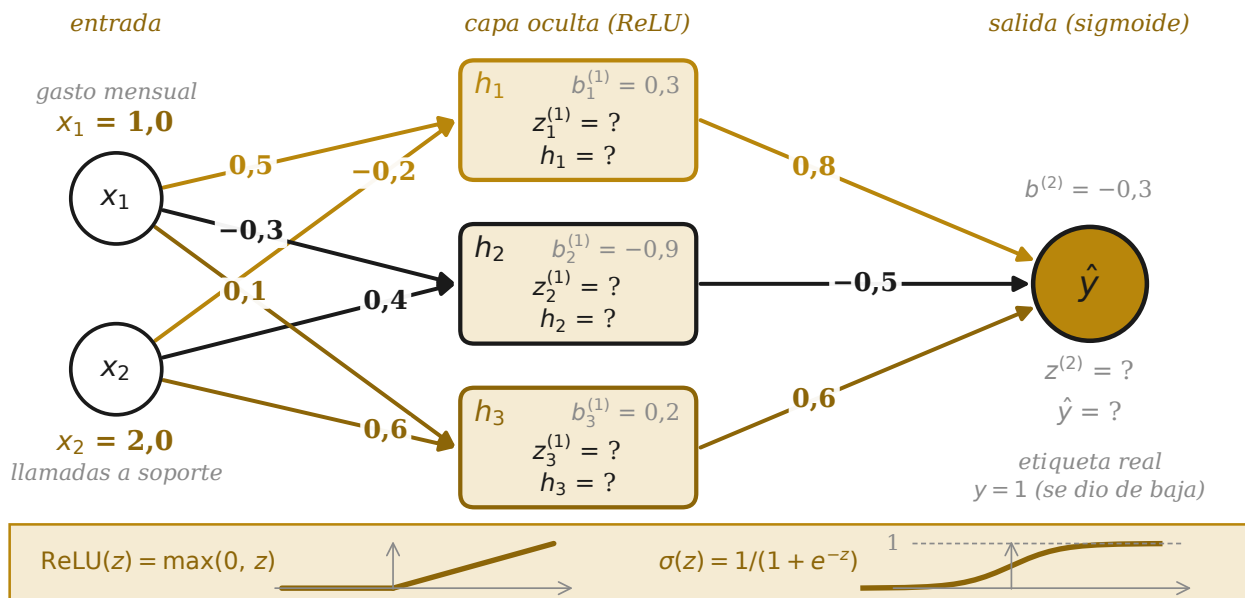
Del cliente 4713 se observa

$$\mathbf{x} = (1,0, 2,0)^\top,$$

es decir, gasta una desviación típica por encima de la media y llama al soporte dos desviaciones típicas por encima de la media. El cliente pertenece al histórico de entrenamiento y efectivamente se dio de baja el trimestre siguiente, de modo que la etiqueta real es $y = 1$. Los parámetros de la red son

$$W^{(1)} = \begin{pmatrix} 0,5 & -0,2 \\ -0,3 & 0,4 \\ 0,1 & 0,6 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 0,3 \\ -0,9 \\ 0,2 \end{pmatrix}, \quad \mathbf{w}^{(2)} = \begin{pmatrix} 0,8 \\ -0,5 \\ 0,6 \end{pmatrix}, \quad b^{(2)} = -0,3,$$

donde la fila j de $W^{(1)}$ contiene los pesos de la neurona oculta j , de manera que $W_{jk}^{(1)}$ es el peso con el que la entrada x_k entra en la neurona oculta j .



Las mismas matrices, dibujadas. Cada arista lleva su peso y va del color de la neurona a la que llega (igual que las filas de $W^{(1)}$ arriba): la fila j son las dos aristas que entran en h_j , y la columna k , las tres que salen de x_k . Los sesgos entran en la suma sin multiplicar por nada. Rellena lo que aparece con “?”.

- Paso 1.** Calcula las tres pre-activaciones de la capa oculta, $\mathbf{z}^{(1)} = W^{(1)} \mathbf{x} + \mathbf{b}^{(1)}$.
- Paso 2.** Aplica la activación: $\mathbf{h} = \text{ReLU}(\mathbf{z}^{(1)})$, con $\text{ReLU}(z) = \max(0, z)$ componente a componente.
- Paso 3.** Calcula la pre-activación de la salida, $z^{(2)} = \mathbf{w}^{(2)\top} \mathbf{h} + b^{(2)}$.
- Paso 4.** Aplica la sigmoide $\sigma(z) = 1/(1 + e^{-z})$ para obtener la predicción $\hat{y}$.

- e) **Paso 5.** Calcula el error final con la entropía cruzada binaria, $\mathcal{L} = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$.
- f) **Lectura de negocio.** ¿Se marca al cliente como *churn* con umbral 0,5? ¿Acertó la red? ¿Cuánto habría valido el error si el cliente *no* se hubiera dado de baja ($y = 0$)? Identifica qué neurona oculta no aporta nada al veredicto de este cliente y qué variable domina la decisión.

Solución

a) Paso 1 — pre-activaciones de la capa oculta.

Cada neurona oculta hace su propia combinación lineal de las dos entradas y le suma su sesgo:

$$\begin{aligned} z_1^{(1)} &= W_{11}^{(1)} x_1 + W_{12}^{(1)} x_2 + b_1^{(1)} = (0,5)(1,0) + (-0,2)(2,0) + 0,3 = 0,5 - 0,4 + 0,3 = 0,4, \\ z_2^{(1)} &= W_{21}^{(1)} x_1 + W_{22}^{(1)} x_2 + b_2^{(1)} = (-0,3)(1,0) + (0,4)(2,0) + (-0,9) = -0,3 + 0,8 - 0,9 = -0,4, \\ z_3^{(1)} &= W_{31}^{(1)} x_1 + W_{32}^{(1)} x_2 + b_3^{(1)} = (0,1)(1,0) + (0,6)(2,0) + 0,2 = 0,1 + 1,2 + 0,2 = 1,5. \end{aligned}$$

En forma vectorial, $\mathbf{z}^{(1)} = (0,4, -0,4, 1,5)^\top$.

b) Paso 2 — activación ReLU.

$$h_1 = \text{ReLU}(0,4) = 0,4, \quad h_2 = \text{ReLU}(-0,4) = 0, \quad h_3 = \text{ReLU}(1,5) = 1,5,$$

es decir, $\mathbf{h} = (0,4, 0, 1,5)^\top$. La segunda neurona tiene pre-activación negativa y ReLU la apaga.

c) Paso 3 — pre-activación de la salida.

$$\begin{aligned} z^{(2)} &= w_1^{(2)} h_1 + w_2^{(2)} h_2 + w_3^{(2)} h_3 + b^{(2)} \\ &= (0,8)(0,4) + (-0,5)(0) + (0,6)(1,5) + (-0,3) \\ &= 0,32 + 0 + 0,90 - 0,30 \\ &= 0,92. \end{aligned}$$

d) Paso 4 — sigmoide.

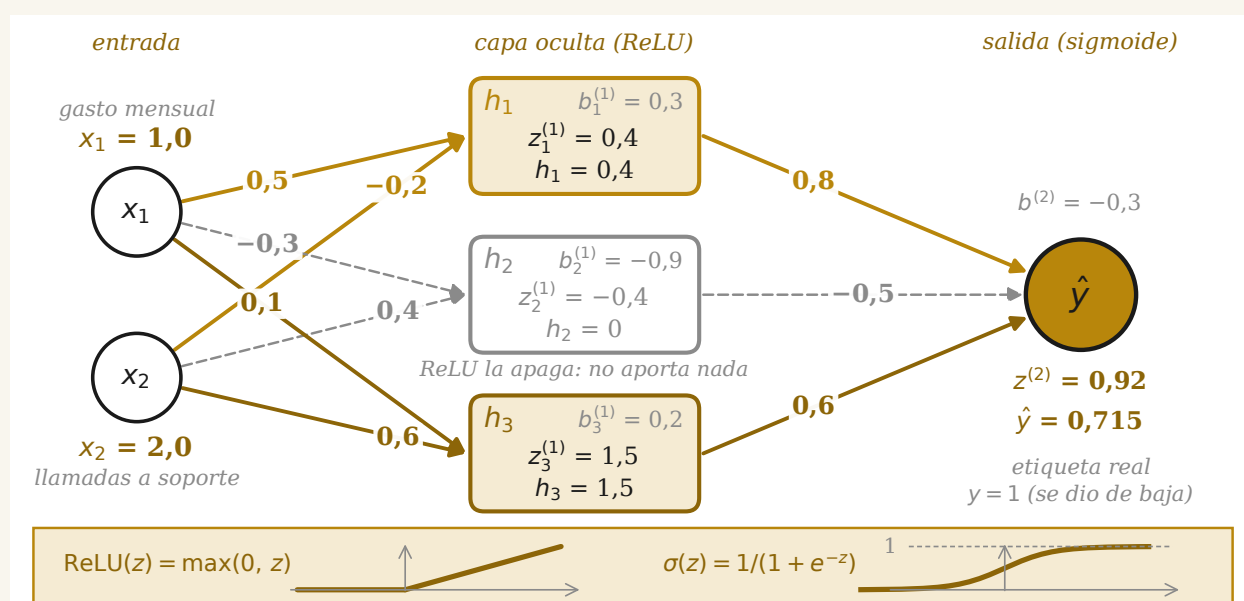
$$\hat{y} = \sigma(0,92) = \frac{1}{1 + e^{-0,92}} = \frac{1}{1 + 0,3985} = \frac{1}{1,3985} \approx 0,7150.$$

e) Paso 5 — error final.

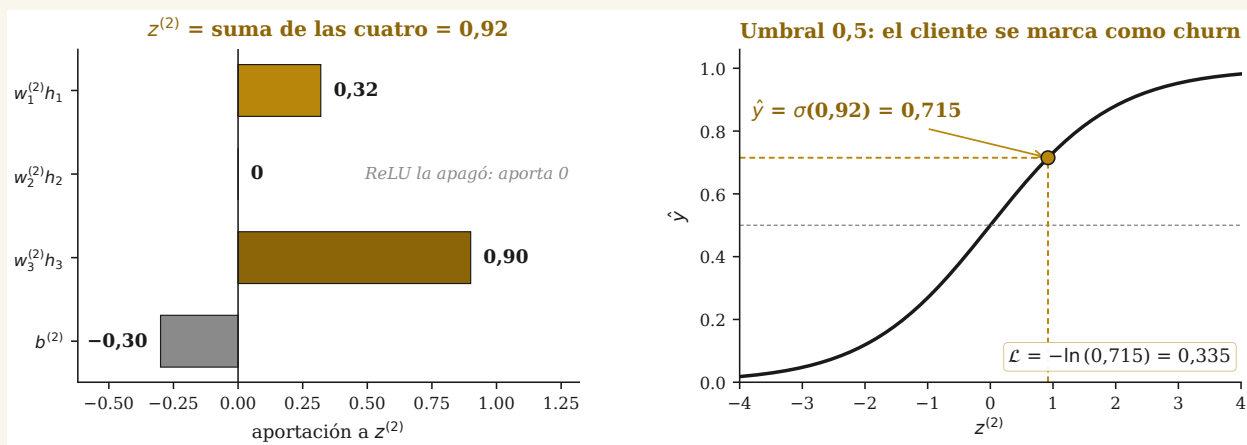
Con $y = 1$ el segundo término de la entropía cruzada se anula y queda

$$\mathcal{L} = -[1 \cdot \ln(0,7150) + 0 \cdot \ln(0,2850)] = -\ln(0,7150) \approx 0,335.$$

El error no es cero aunque la predicción vaya en la dirección correcta: la red solo asigna un 71,5 % de probabilidad al desenlace que de hecho ocurrió, y la entropía cruzada cobra por esa diferencia. Solo con $\hat{y} \rightarrow 1$ se tendría $\mathcal{L} \rightarrow 0$.



El pase completo sobre el mismo dibujo. Cada caja lleva ya su pre-activación y su activación. La neurona 2 sale en gris y con las aristas de puntos: su pre-activación es negativa, ReLU la apaga y su aportación a la salida es exactamente 0; para este cliente la red es, de hecho, una red $2 \rightarrow 2 \rightarrow 1$.



De dónde sale el 0,92 y qué hace la sigmoide con él. A la izquierda, las cuatro aportaciones que se suman en $z^{(2)}$: los 0,90 de la tercera neurona pesan casi el triple que los 0,32 de la primera, la segunda no aporta nada y el sesgo resta 0,30. A la derecha, la sigmoide lleva ese 0,92 a $\hat{y} = 0,715$, por encima del umbral 0,5; el error todavía vale 0,335 porque a la etiqueta que ocurrió sólo se le asigna un 71,5 % de probabilidad.

f) Lectura de negocio.

Con $\hat{y} \approx 0,7150 > 0,5$ el cliente se marca como *churn*, y como $y = 1$ la clasificación es correcta: sobre este cliente del histórico el modelo habría disparado la campaña de retención a tiempo. Si el cliente no se hubiera dado de baja ($y = 0$), la predicción seguiría siendo 0,7150 pero el error sería

$$\mathcal{L} = -\ln(1 - 0,7150) = -\ln(0,2850) \approx 1,255,$$

casi cuatro veces mayor: la misma salida es un acierto tibio o un fallo caro según la etiqueta.

La neurona oculta que no aporta nada es la segunda: $z_2^{(1)} = -0,4 < 0$, así que $h_2 = 0$ y su contribución a la salida es $w_2^{(2)}h_2 = (-0,5)(0) = 0$. Para este cliente la red funciona, de hecho, como una red $2 \rightarrow 2 \rightarrow 1$. Desglosando $z^{(2)}$ por sumandos, la tercera neurona aporta 0,90 frente a los 0,32 de la primera, y dentro de $z_3^{(1)} = 1,5$ el término grande es $(0,6)(2,0) = 1,2$, que viene de x_2 . Son las llamadas al soporte, no el gasto, las que dominan el veredicto.

2. BatchNorm paso a paso sobre un minibatch de cuatro pedidos

Día 03 · Tema 1 — regularización y learning rate schedules.

Una cadena de moda entrena una red que predice si un pedido acabará en devolución. Una neurona de la primera capa oculta reacciona sobre todo al importe del pedido, y su pre-activación sobre el minibatch de cuatro pedidos que toca en esta iteración es

$$\mathbf{a} = (3,0, 9,0, 11,0, 17,0),$$

donde a_i es la pre-activación del pedido i : P_1 es el pedido de importe más bajo del lote y P_4 el más alto. Detrás de esa neurona hay una capa de *Batch Normalization* con parámetros ya aprendidos $\gamma = 1,2$ y $\beta = 0,3$.

En entrenamiento, *BatchNorm* normaliza con la media y la varianza del *propio minibatch* (varianza con divisor n , no $n - 1$) y después aplica una transformación afín aprendida:

$$\mu_B = \frac{1}{n} \sum_{i=1}^n a_i, \quad \sigma_B^2 = \frac{1}{n} \sum_{i=1}^n (a_i - \mu_B)^2, \quad \hat{a}_i = \frac{a_i - \mu_B}{\sigma_B}, \quad y_i = \gamma \hat{a}_i + \beta.$$

- Paso 1.** Calcula la media μ_B , la varianza σ_B^2 y la desviación típica σ_B del minibatch.
- Paso 2.** Normaliza los cuatro valores para obtener $\hat{\mathbf{a}}$.
- Paso 3.** Aplica la transformación afín para obtener $\mathbf{y}$.
- Lectura de negocio.** Si esa neurona hubiera venido midiendo el importe en céntimos en lugar de en euros, sus cuatro pre-activaciones habrían sido (300, 900, 1 100, 1 700). Recalcula μ_B , σ_B y $\hat{\mathbf{a}}$ con esos valores. ¿Qué recibe la capa siguiente en cada caso? ¿Qué está midiendo entonces la red después de la capa de *BatchNorm*, euros o algo distinto?

Solución

a) Paso 1 — media y varianza del minibatch.

Con $n = 4$:

$$\mu_B = \frac{3,0 + 9,0 + 11,0 + 17,0}{4} = \frac{40,0}{4} = 10,0.$$

Las desviaciones respecto a la media son $-7, -1, +1$ y $+7$, de modo que

$$\sigma_B^2 = \frac{(-7)^2 + (-1)^2 + (1)^2 + (7)^2}{4} = \frac{49 + 1 + 1 + 49}{4} = \frac{100}{4} = 25,0, \quad \sigma_B = \sqrt{25,0} = 5,0.$$

Cuidado con el divisor: la varianza *muestral* (divisor $n - 1 = 3$) valdría $100/3 \approx 33,33$ y daría $\sigma \approx 5,77$. *BatchNorm* normaliza con el divisor n , y es lo que hace que $\hat{\mathbf{a}}$ tenga varianza exactamente 1.

b) Paso 2 — normalizar.

Dividiendo cada desviación por $\sigma_B = 5$:

$$\begin{aligned} \hat{a}_1 &= \frac{3 - 10}{5} = \frac{-7}{5} = -1,4, \\ \hat{a}_2 &= \frac{9 - 10}{5} = \frac{-1}{5} = -0,2, \\ \hat{a}_3 &= \frac{11 - 10}{5} = \frac{1}{5} = 0,2, \\ \hat{a}_4 &= \frac{17 - 10}{5} = \frac{7}{5} = 1,4. \end{aligned}$$

Es decir, $\hat{\mathbf{a}} = (-1,4, -0,2, 0,2, 1,4)$. Su media es 0 y su varianza es $(1,96 + 0,04 + 0,04 + 1,96)/4 = 1$, como debía ser: eso es lo único que hace el paso de normalizar.

c) Paso 3 — escalar y desplazar.

Con $\gamma = 1,2$ y $\beta = 0,3$:

$$\begin{aligned} y_1 &= 1,2 \times (-1,4) + 0,3 = -1,68 + 0,30 = -1,38, \\ y_2 &= 1,2 \times (-0,2) + 0,3 = -0,24 + 0,30 = 0,06, \\ y_3 &= 1,2 \times 0,2 + 0,3 = 0,24 + 0,30 = 0,54, \\ y_4 &= 1,2 \times 1,4 + 0,3 = 1,68 + 0,30 = 1,98. \end{aligned}$$

La media de $\mathbf{y}$ es $(-1,38 + 0,06 + 0,54 + 1,98)/4 = 0,30 = \beta$ y su desviación típica es $1,2 = \gamma$: el centro y la escala con que la señal entra en la capa siguiente son ahora dos *parámetros aprendidos*, no un accidente.

d) Lectura de negocio.

Con los importes en céntimos, $\mathbf{a}' = 100 \mathbf{a} = (300, 900, 1 100, 1 700)$:

$$\mu'_B = \frac{300 + 900 + 1 100 + 1 700}{4} = 1 000, \quad \sigma'^2_B = \frac{700^2 + 100^2 + 100^2 + 700^2}{4} = 250 000, \quad \sigma'_B = 500,$$

$$\hat{a}'_1 = \frac{300 - 1\,000}{500} = -1,4, \quad \hat{a}'_2 = \frac{900 - 1\,000}{500} = -0,2, \quad \hat{a}'_3 = 0,2, \quad \hat{a}'_4 = 1,4.$$

Sale *exactamente* el mismo vector que en el apartado b), y por tanto la misma y . No es casualidad: multiplicar los datos por una constante $c > 0$ multiplica por c tanto μ_B como σ_B , y el cociente $(a_i - \mu_B)/\sigma_B$ no se mueve.

La lectura es que, después de la capa, la red ya no está mirando euros ni céntimos: está mirando *cuántas desviaciones típicas se sale este pedido del pedido medio de su lote*, una cifra sin unidades. P_1 se sale 1,4 desviaciones por abajo y P_2 solo 0,2; eso es lo que la siguiente capa usa para decidir, y no cambiaría si mañana el sistema empezase a enviar los importes en céntimos, en miles de euros o en dólares.

Esto es lo que hay detrás del “permite un *learning rate* más alto” de la teoría. Sin normalizar, una *feature* que llega en céntimos produce gradientes cien veces mayores que la misma *feature* en euros, y hay que bajar el η hasta que la peor escala del modelo no lo desestabilice. Normalizando, todas las capas ven señales del mismo tamaño, la escala final la fija la red con γ y β , y se puede entrenar más rápido con menos tiempo perdido en ajustar hiperparámetros: menos horas de GPU y menos horas de analista por modelo entregado.

3. Convolución y pooling a mano: el detector de huecos de estante

Día 04 · Tema 2 — convolución, pooling y transfer learning.

Una cadena de supermercados instala en sus tiendas una app que fotografía los estantes y avisa cuando hay un *hueco*, es decir, un espacio vacío donde debería haber producto. El primer filtro de la CNN que va dentro de la app busca *bordes verticales*, porque la frontera entre el bloque de producto y el hueco vacío es exactamente una línea vertical.

Trabajamos sobre un recorte de 4×4 píxeles de una de esas fotos, en escala de grises, donde 0 es negro y 9 es blanco:

$$\mathbf{X} = \begin{pmatrix} 9 & 8 & 1 & 1 \\ 8 & 9 & 1 & 2 \\ 6 & 5 & 6 & 5 \\ 5 & 4 & 6 & 6 \end{pmatrix}.$$

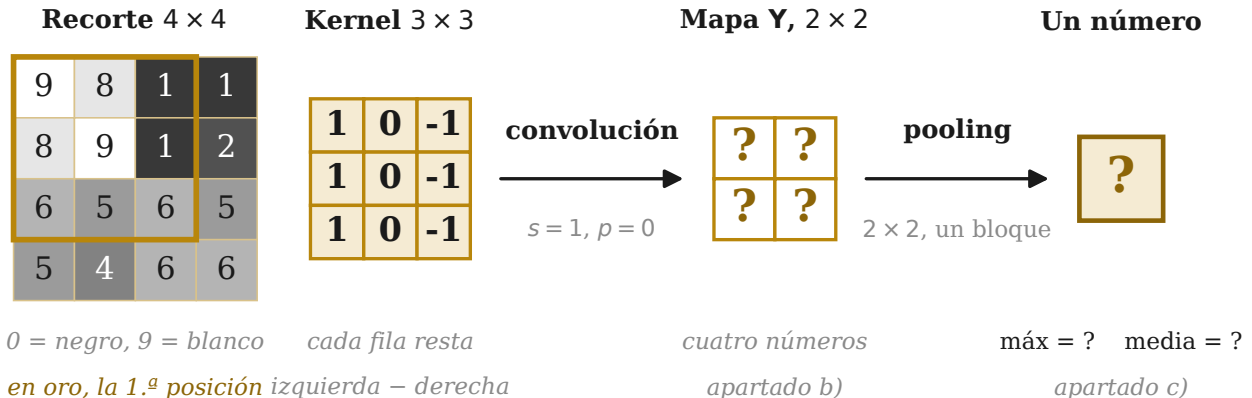
En la mitad de arriba las dos columnas de la izquierda son claras (producto iluminado) y las dos de la derecha oscuras (el hueco vacío, que no tiene producto que refleje la luz). La mitad de abajo es gris intermedio en toda su anchura: ahí hay producto de lado a lado y no hay hueco.

El kernel del filtro es el detector de bordes verticales de las slides:

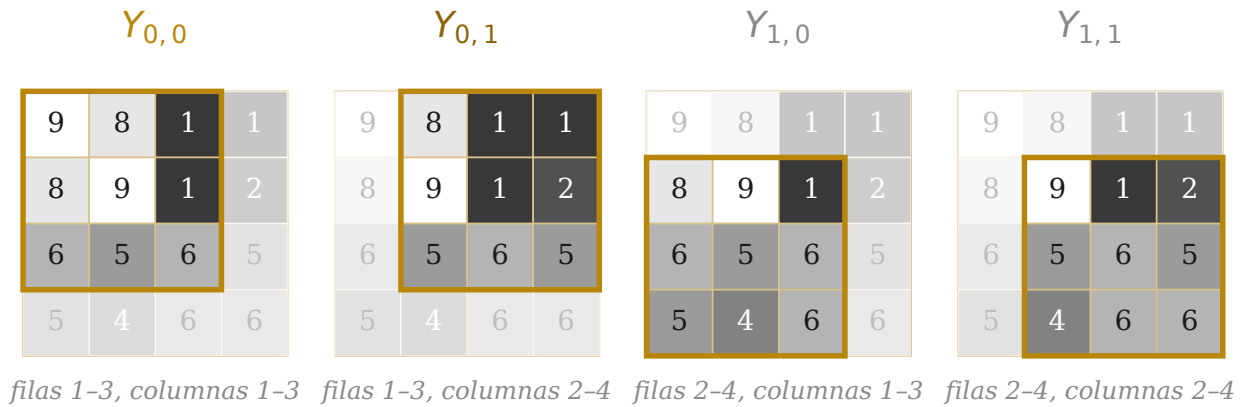
$$\mathbf{K} = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}.$$

Leído en palabras: en cada una de sus tres filas resta el píxel de la derecha al de la izquierda e ignora el del centro. Devuelve un número grande y positivo cuando el parche es claro a la izquierda y oscuro a la derecha, y cero cuando el parche es plano.

La capa convolucional va sin *padding* ($p = 0$), con *stride* $s = 1$, sin sesgo y sin función de activación. Detrás de ella va una capa de *pooling* de ventana 2×2 y *stride* 2.



Lo que hay que rellenar, de un vistazo. Los píxeles del recorte están pintados con su nivel de gris real: el bloque oscuro de la mitad superior derecha es el hueco del estante. El kernel se coloca sobre un parche 3×3 de la imagen (en oro, la primera de las cuatro posiciones), y cada posición produce una casilla del mapa Y. La capa de pooling se come el mapa entero y deja un solo número.



Qué parche le toca a cada casilla del mapa. Las cuatro posiciones del kernel, en el orden del apartado (b). En cada una, los píxeles velados son los que *no* entran en esa cuenta: para $Y_{0,0}$ sobra la última fila y la última columna, para $Y_{1,1}$ sobran la primera fila y la primera columna, y así. Cada recuadro dorado son los nueve números que se multiplican por el kernel.

a) **Paso 1: tamaño del mapa de salida.** Con la fórmula

$$H' = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

(y la análoga para W'), di cuántas filas y cuántas columnas tiene el mapa de activación Y que sale de la convolución, es decir, en cuántas posiciones distintas cabe el kernel sobre la imagen. Aquí $H = W = 4$, $k = 3$, $p = 0$ y $s = 1$. Todavía no hace falta calcular ningún valor.

b) **Paso 2: los valores del mapa.** Calcula las cuatro casillas de Y con

$$Y_{i,j} = \sum_{u=0}^2 \sum_{v=0}^2 X_{i+u, j+v} K_{u,v}, \quad i, j \in \{0, 1\},$$

que en la práctica significa colocar el kernel sobre un parche 3×3 de la imagen, multiplicar casilla a casilla y sumar los nueve productos. Los cuatro parches son, en orden: $Y_{0,0}$ usa las tres primeras filas y las tres primeras columnas de X ; $Y_{0,1}$, las tres primeras filas y las tres últimas columnas; $Y_{1,0}$, las tres últimas filas y las tres primeras columnas; y $Y_{1,1}$, las tres últimas filas y las tres últimas columnas. Escribe el resultado como una matriz 2×2 .

c) **Paso 3: pooling.** La ventana 2×2 de la capa de pooling cubre el mapa Y entero en un solo bloque, de modo que esta capa devuelve un único número, que es el que llega al clasificador final.

- i) ¿Qué número sale con *max-pooling* (el máximo del bloque)?
- ii) ¿ Y con *average-pooling* (la media del bloque)?
- iii) ¿Cuántos parámetros entrenables añade a la red esta capa de pooling?

d) **Lectura de negocio.** Ese único número es toda la evidencia de «hay un borde vertical en esta foto» con la que se queda la red. Primero: de los dos resúmenes del apartado (c), ¿cuál mandarías a producción para disparar la alarma de hueco, y qué está diluyendo el otro? Justifícalo señalando qué casillas de Y vienen de la mitad de arriba de la foto y cuáles de la mitad de abajo. Segundo: si el mismo estante se fotografía un poco más abajo, de forma que la frontera producto-hueco cae en otras filas del recorte, ¿cambia mucho el número que sale del max-pooling? ¿Qué propiedad le da eso a la app que un MLP sobre los píxeles en crudo no tendría?

Solución

a) Paso 1 – tamaño del mapa de salida.

Con $H = W = 4$, $k = 3$, $p = 0$ y $s = 1$:

$$H' = \left\lfloor \frac{4 + 2 \cdot 0 - 3}{1} \right\rfloor + 1 = \lfloor 1 \rfloor + 1 = 2, \quad W' = 2.$$

El mapa es $Y \in \mathbb{R}^{2 \times 2}$: el kernel 3×3 cabe en cuatro posiciones sobre la imagen 4×4 (dos horizontales por dos verticales), y cada posición produce un número. La convolución sin padding encoge la imagen: de 16 píxeles a 4 activaciones.

b) Paso 2 – los cuatro valores.

Como el kernel tiene la columna central a cero, cada una de sus tres filas aporta simplemente (píxel izquierdo – píxel derecho) del parche. Conviene calcularlo así, fila a fila, y luego sumar las tres.

$Y_{0,0}$: parche formado por las tres primeras filas y las tres primeras columnas,

$$\begin{pmatrix} 9 & 8 & 1 \\ 8 & 9 & 1 \\ 6 & 5 & 6 \end{pmatrix} \Rightarrow (9 - 1) + (8 - 1) + (6 - 6) = 8 + 7 + 0 = 15.$$

$Y_{0,1}$: tres primeras filas, tres últimas columnas,

$$\begin{pmatrix} 8 & 1 & 1 \\ 9 & 1 & 2 \\ 5 & 6 & 5 \end{pmatrix} \Rightarrow (8 - 1) + (9 - 2) + (5 - 5) = 7 + 7 + 0 = 14.$$

$Y_{1,0}$: tres últimas filas, tres primeras columnas,

$$\begin{pmatrix} 8 & 9 & 1 \\ 6 & 5 & 6 \\ 5 & 4 & 6 \end{pmatrix} \Rightarrow (8 - 1) + (6 - 6) + (5 - 6) = 7 + 0 - 1 = 6.$$

$Y_{1,1}$: tres últimas filas, tres últimas columnas,

$$\begin{pmatrix} 9 & 1 & 2 \\ 5 & 6 & 5 \\ 4 & 6 & 6 \end{pmatrix} \Rightarrow (9 - 2) + (5 - 5) + (4 - 6) = 7 + 0 - 2 = 5.$$

Es decir,

$$Y = \begin{pmatrix} 15 & 14 \\ 6 & 5 \end{pmatrix}.$$

La lectura del mapa ya se ve a simple vista: la fila de arriba de Y (15 y 14) corresponde a los parches que contienen las dos filas claras sobre oscuro, donde está la frontera del hueco; la fila de abajo (6 y 5) corresponde a los parches dominados por la zona gris uniforme, donde el filtro casi no encuentra nada. El filtro no ha aprendido «hueco»: ha medido «cambio brusco de claro a oscuro en horizontal», que es la pista de la que se alimenta el clasificador.

c) Paso 3 – pooling.

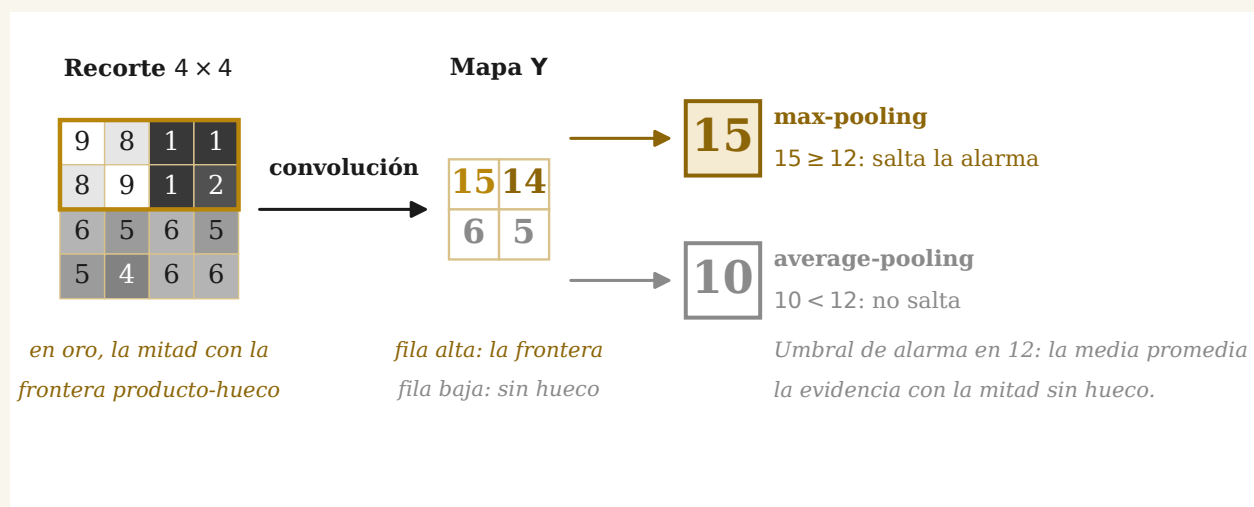
i) *Max-pooling*. El único bloque 2×2 es el mapa completo, y su máximo es

$$\max\{15, 14, 6, 5\} = 15.$$

ii) *Average-pooling*. La media del mismo bloque es

$$\frac{15 + 14 + 6 + 5}{4} = \frac{40}{4} = 10.$$

iii) *Parámetros. Cero*. El pooling no tiene pesos ni sesgos: es una regla fija (tomar el máximo, o la media) sin nada que aprender. Es la capa más barata de la red y, sin embargo, la que divide por cuatro el número de activaciones que llegan a las capas siguientes, y con ello el tamaño de la capa densa final.



La cadena entera, con los números. La mitad de arriba del recorte, la que contiene la frontera producto-hueco, produce las dos casillas altas del mapa (15 y 14); la mitad de abajo, gris uniforme, produce las dos bajas (6 y 5). Con un umbral de alarma en 12, el max-pooling deja pasar el 15 y la app avisa; el average-pooling entrega un 10 y el hueco se queda sin detectar. Es el apartado (d) en una imagen.

d) Lectura de negocio.

Qué resumen va a producción. El máximo. Lo que la app necesita saber es si hay un hueco *en alguna parte* del estante, no cuánto hueco hay de media. Las dos casillas de arriba de Y (15 y 14) son las que contienen la frontera producto-hueco; las dos de abajo (6 y 5) vienen de la zona gris uniforme, donde no hay nada que detectar. El max-pooling se queda con el 15 y la alarma se dispara; la media promedia los aciertos con los ceros y baja la evidencia a 10, es decir, diluye la señal con las tres cuartas partes de la foto donde no pasa nada. Con un umbral de alarma fijado, por ejemplo, en 12, el mismo estante se detectaría con max-pooling y se dejaría pasar con average-pooling. Ese es el sesgo típico del average: en detección de eventos raros y localizados, promediar sobre toda la imagen es regalar falsos negativos, y en este negocio el falso negativo es la venta perdida.

Qué pasa si la foto se desplaza. Casi nada. En una foto real la frontera producto-hueco es una línea vertical que recorre el estante de arriba abajo, así que si el recorte se toma un poco más abajo el filtro se encuentra el mismo salto de claro a oscuro, solo que registrado en otra casilla del mapa: donde antes salía el 15 en $Y_{0,0}$ ahora saldrá un valor parecido en $Y_{1,0}$, y el máximo del bloque apenas se mueve. Esa es la propiedad que se llama *invarianza a pequeñas traslaciones*, y viene de la combinación de dos cosas: el kernel se reutiliza en todas las posiciones (pesos compartidos), y el pooling se queda con la evidencia más fuerte sin importar dónde estaba. Un MLP sobre los píxeles en crudo tiene un peso distinto para cada píxel, así que la misma foto desplazada dos filas activa un conjunto de pesos completamente diferente y hay que enseñarle el hueco otra vez en cada posición posible. Traducido a negocio: con la CNN el empleado puede apuntar el móvil a ojo, y con el MLP habría que fijar la cámara y recalibrar cada tienda.